

Sunburn device: the whole program

micro:bit firmware 3.1 · the whole program, block by block · printed 2026-09-14

This is the complete MakeCode program running on the micro:bit, exactly as it appears in the editor. This sheet is the whole workspace at a glance: nine numbered sections in two rows. The other sheets show every block of each section at readable size, with what it does, what it needs, and which other blocks use it. Sections 1 and 2 are the whole program; sections 3 to 9 are the parts the flowchart calls.



- 1

Start here: settings · 1 block · sheet 2
The on start block. First every setting the program uses (sensor type, servo, screen, UV bands, timings), then the start-up steps: this is "Device turns on" on the flowchart. Change hardware and timings here and nowhere else.
- 4

Alerts · 4 blocks · sheet 8
Flash and beep, tap the arm with the servo, and standby. Each one keeps going until A is pressed (or the app sends "ack"), the device is turned off, or its time limit runs out. No answer within the time limit means the device is probably not being worn, so it goes to standby until a button is pressed.
- 7

Bluetooth and serial · 11 blocks · sheet 11
Talks to the phone app (Bluetooth UART) and to the USB serial console with the same text lines. Out every 2 s:
st=wait;uv=7.3;sen=7.1;onl=6.5;band=vhigh;spf=5400;spfn=3;fw=3.1;demo=0.
Out on events: ev=on, off, sunscreen, reapply, inside, standby, error. In, one per line: uv=6.5, ack, zero, cal=7, demo=1, power=0, debug=1, ping, read.

- 2

The flowchart · 20 blocks · sheet 3
forever runs runFlowchart again and again. runFlowchart is the whole flowchart from top to bottom: read the UV sensor, sensor in range?, phone sending online UV?, UV index?, then one of the four branches, wait 5 minutes, reapply timer expired?. Every box and decision is a function with the same name, in the same order as the flowchart.
- 5

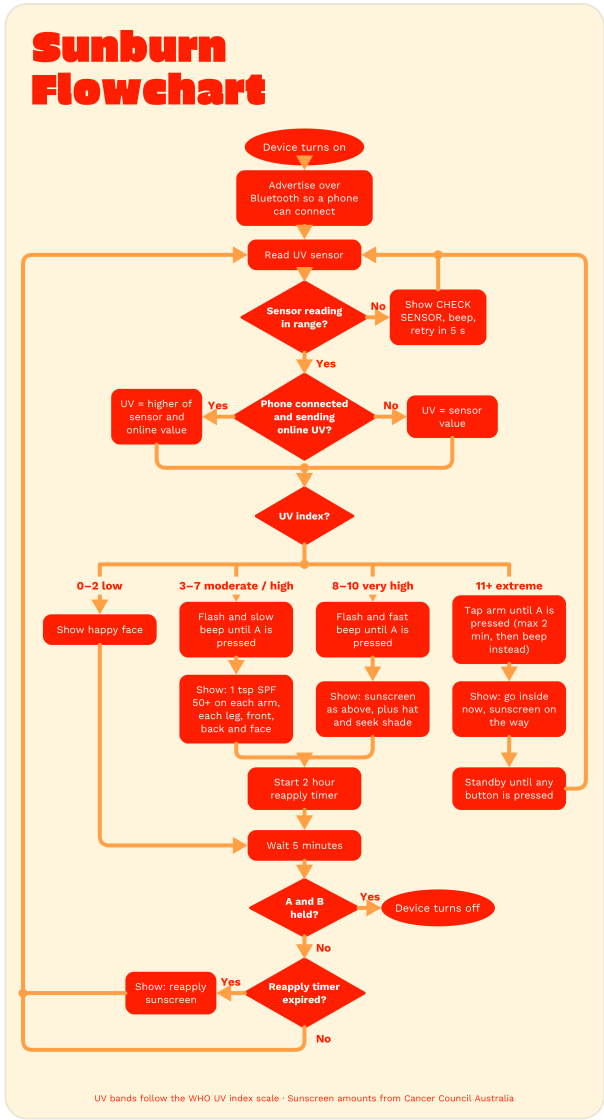
Screen and LEDs · 8 blocks · sheet 9
What the 5x5 LEDs show, and what the optional OLED screen shows: the UV number, the band, messages and the countdowns. Messages scroll across the LEDs when there is no OLED.
- 8

Helpers · 10 blocks · sheet 14
Small tools used by the rest: sensor presets, the timings, sorting, number and time formatting, tidying up commands, and the debug lines (send debug=1 to see them).
- 3

Buttons · 3 blocks · sheet 7
A = done (sunscreen is on / going inside); A also skips a message scrolling on the LEDs, and while waiting it shows the UV index and the countdown. B = wake from standby. A+B = device turns off, or on again.
- 6

Sound and servo · 5 blocks · sheet 11
Beeps on the V2 speaker (or a buzzer on P0), and the servo on P2 whose arm taps the wrist.
- 9

OLED screen driver (optional) · 12 blocks · sheet 16
Only used if you add a 128x64 I2C screen (SSD1306 or SH1106) on P19/P20. Draws text on it without needing an extension. The program detects the screen at start-up and works exactly the same without one.



The Sunburn Flowchart the program follows. Every box is a function in section 2 with the same name.

- Variables: remember a value

Maths
- Functions: one flowchart box each

Text
- Basic: LEDs, pause, show

Arrays: lists
- Input: buttons and pins

Music: beeps
- Loops: repeat

Pins: sensor, servo, OLED
- Logic: if, compare

Bluetooth: the app
- Serial: USB console
- Control: timing

Reading a sheet
Every picture is one block from the editor at 60% size. Under it: the block's name, what kind of block it is, what it does, which blocks use it (Used by) and which blocks it calls (Uses).

1

Start here: settings

1 block

The on start block. First every setting the program uses (sensor type, servo, screen, UV bands, timings), then the start-up steps: this is "Device turns on" on the flowchart. Change hardware and timings here and nowhere else.



on start Runs once when the micro:bit powers up

Every setting the program uses comes first (change your hardware and timings here), then the start-up steps: sensor maths, timings, sound, screen, Bluetooth, then "Device turns on". Hold B while switching on for demo timings.

Uses: deviceTurnsOn, setupSound, advertiseOverBluetooth, applySensorPreset, applyTimings, setupOled

2 The flowchart

20 blocks

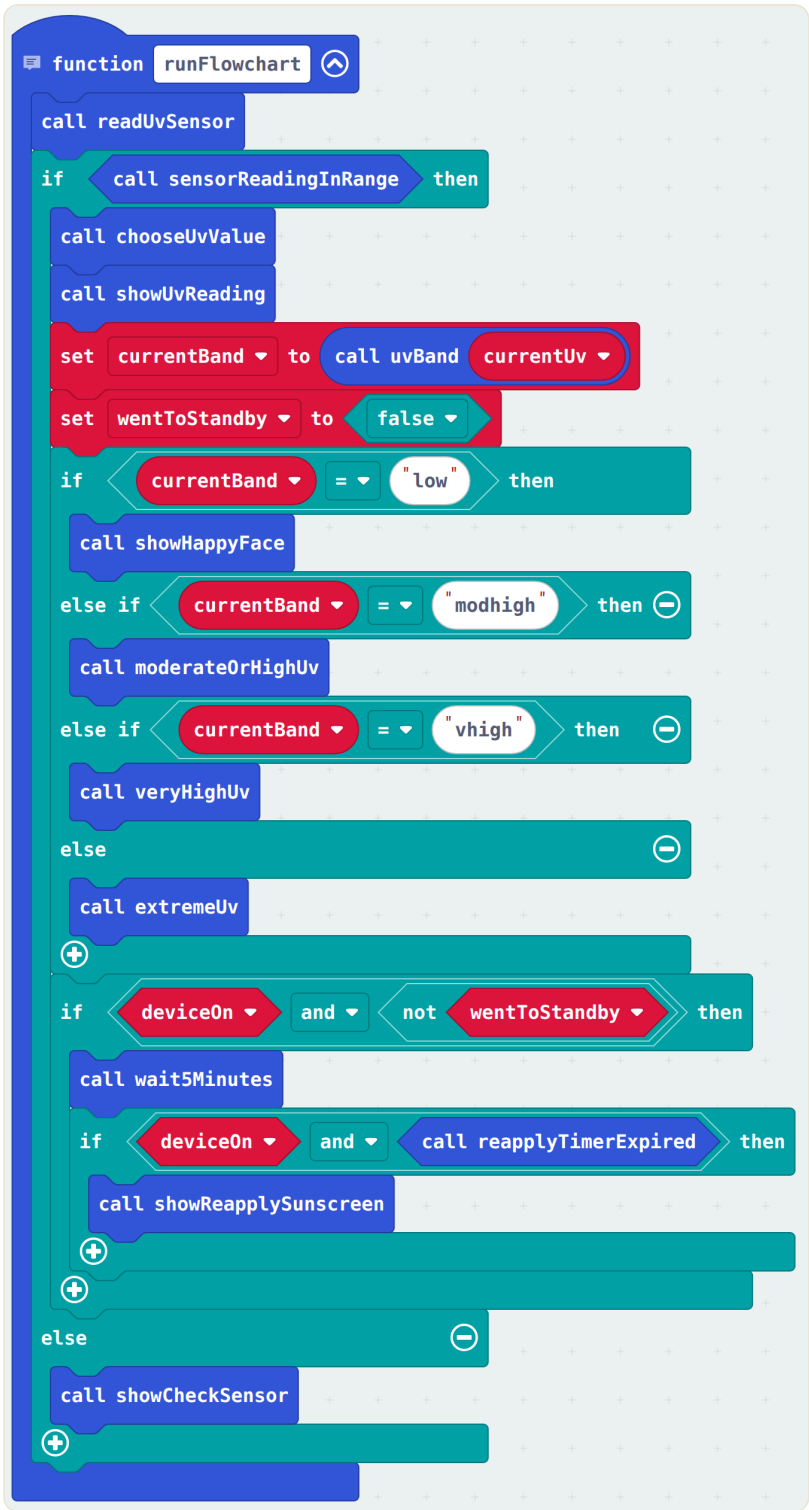
forever runs runFlowchart again and again. runFlowchart is the whole flowchart from top to bottom: read the UV sensor, sensor in range?, phone sending online UV?, UV index?, then one of the four branches, wait 5 minutes, reapply timer expired?. Every box and decision is a function with the same name, in the same order as the flowchart.



forever Runs again and again, for ever

Forever: one trip around the flowchart, then straight back to "Read UV sensor"

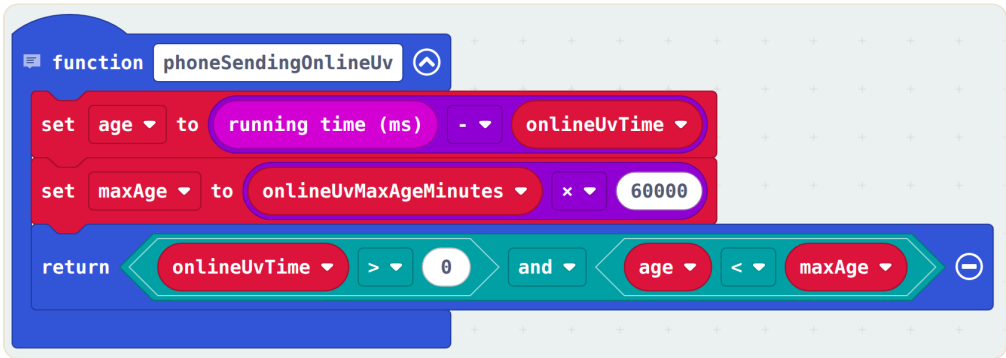
Uses: runFlowchart



runFlowchart Function

The flowchart from top to bottom. Every box and decision is a function below, named after the box. "A and B held?" is the A+B button block: it turns the device off at any moment; every wait stops as soon as that happens and the rest of the trip is skipped. A branch that ended in standby goes straight back to "Read UV sensor" instead of waiting 5 minutes.

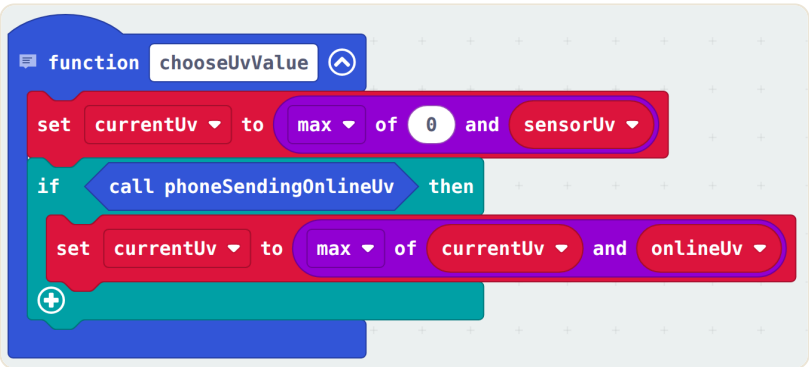
Used by: forever · **Uses:** readUvSensor, sensorReadingInRange, showCheckSensor, chooseUvValue, uvBand, showHappyFace, moderateOrHighUv, veryHighUv, extremeUv, wait5Minutes, reapplyTimerExpired, showReapplySunscreen, showUvReading



phoneSendingOnlineUv Function that answers with a yes/no

Decision: Phone connected and sending online UV? Yes while a UV value from the app (over Bluetooth or USB) arrived in the last 30 minutes.

Used by: chooseUvValue, showUvReading, sendStatus



chooseUvValue Function

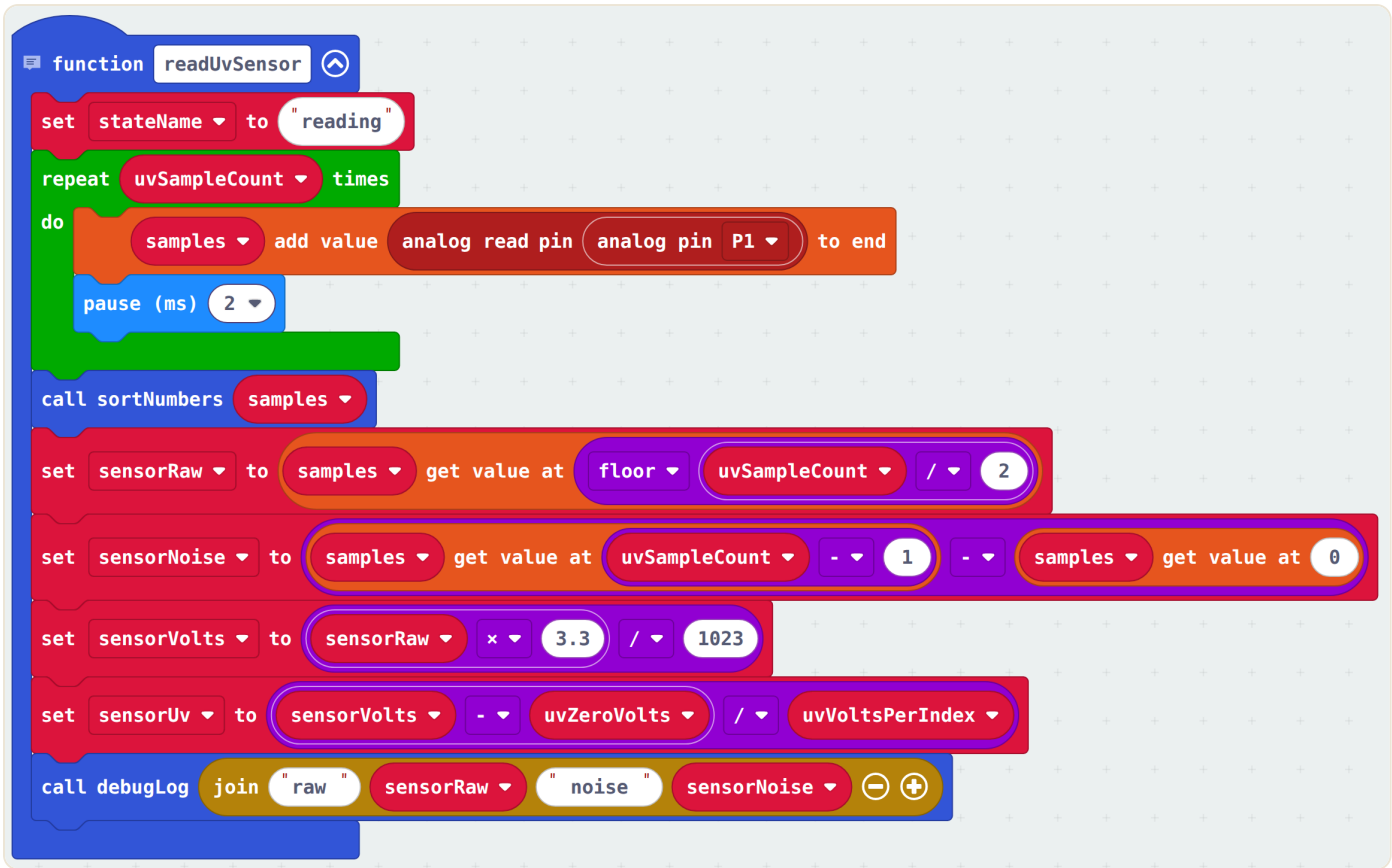
Boxes: UV = higher of sensor and online / UV = sensor value

Used by: runFlowchart · **Uses:** phoneSendingOnlineUv

2 The flowchart (continued)

20 blocks

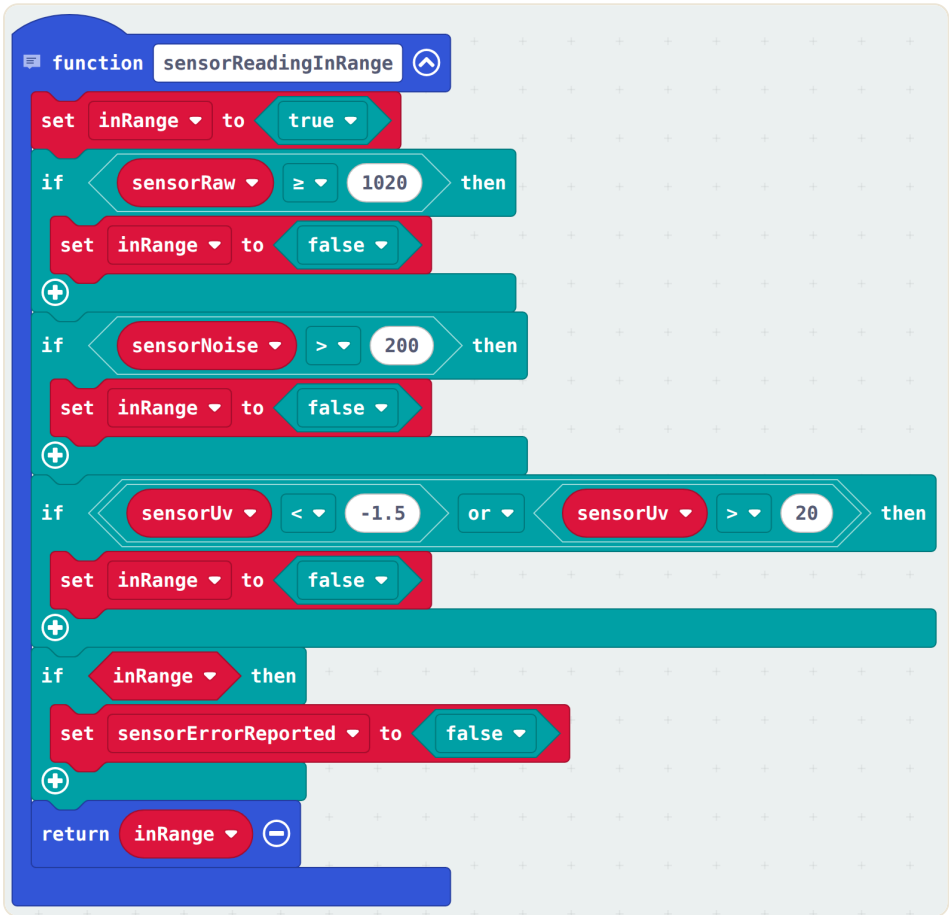
forever runs runFlowchart again and again. runFlowchart is the whole flowchart from top to bottom: read the UV sensor, sensor in range?, phone sending online UV?, UV index?, then one of the four branches, wait 5 minutes, reapply timer expired?. Every box and decision is a function with the same name, in the same order as the flowchart.



readUvSensor Function

Box: Read UV sensor. Analog pin P1. Takes the middle value of several samples so one noisy sample cannot set off a false alarm.

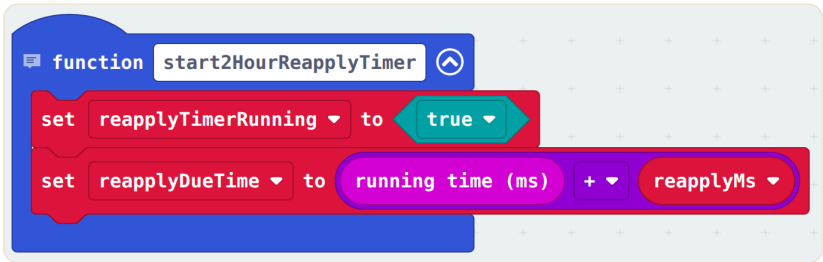
Used by: runFlowchart, handleCommand, calibrateToKnownUv · **Uses:** sortNumbers, debugLog



sensorReadingInRange Function that answers with a yes/no

Decision: Sensor reading in range? No when the pin is stuck at the top of its range (sensor missing or shorted), the samples jump about (loose wire), or the UV number cannot be real.

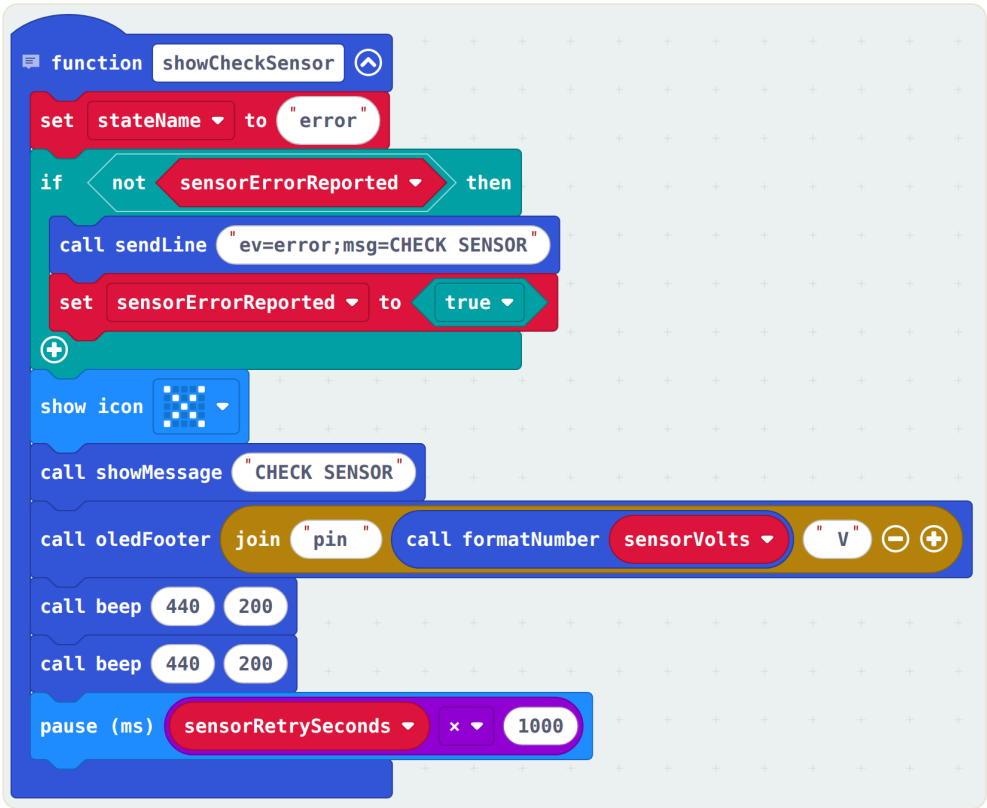
Used by: runFlowchart



start2HourReapplyTimer Function

Box: Start 2 hour reapply timer

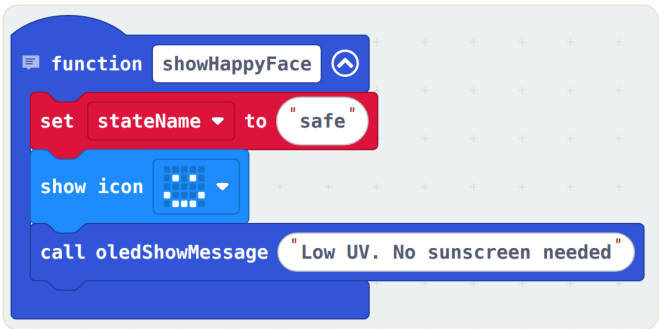
Used by: sunscreenApplied



showCheckSensor Function

Box: Show CHECK SENSOR, beep, retry in 5 s. A bad sensor is never treated as "safe". The app gets one ev=error per problem, not one per retry.

Used by: runFlowchart · **Uses:** showMessage, oledFooter, beep, sendLine, formatNumber



showHappyFace Function

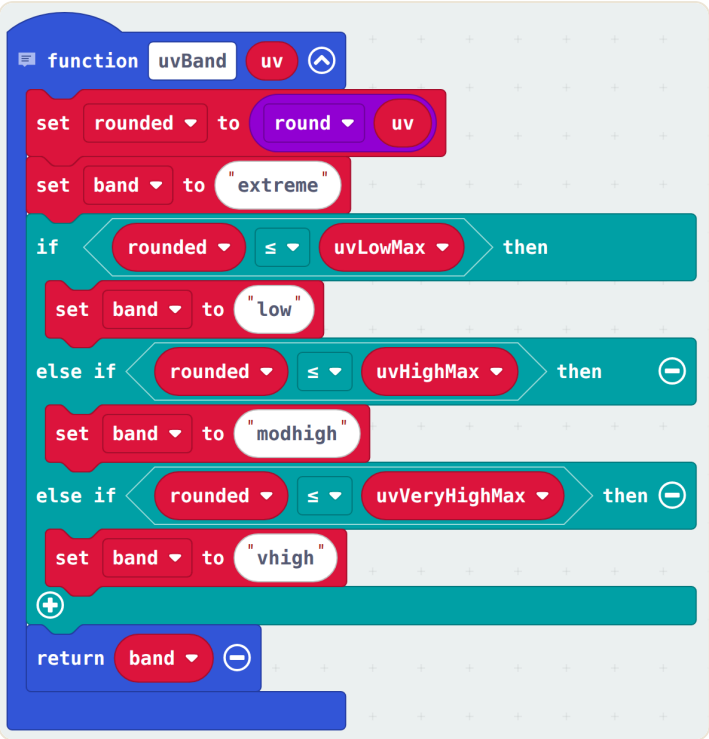
Branch 0-2 low: Show happy face

Used by: runFlowchart · **Uses:** oledShowMessage

2 The flowchart (continued)

20 blocks

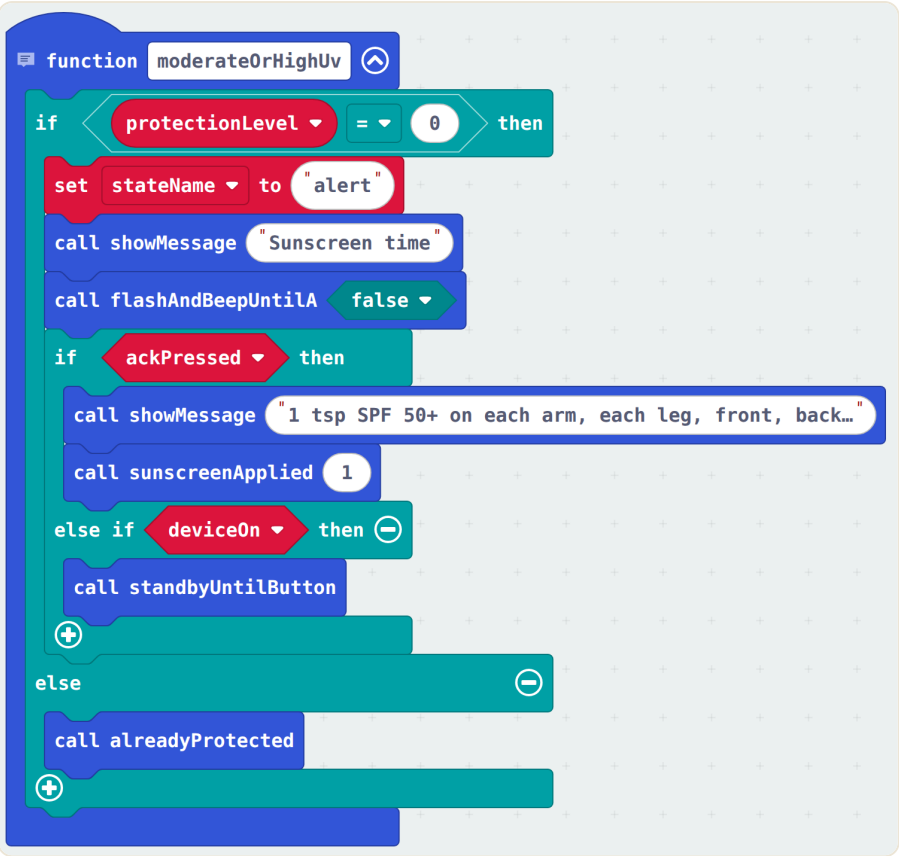
forever runs runFlowchart again and again. runFlowchart is the whole flowchart from top to bottom: read the UV sensor, sensor in range?, phone sending online UV?, UV index?, then one of the four branches, wait 5 minutes, reapply timer expired?. Every box and decision is a function with the same name, in the same order as the flowchart.



uvBand Function that answers with a string, inputs: uv (number)

Decision: UV index? Returns the flowchart branch for a UV index: "low" (0-2), "modhigh" (3-7), "vhigh" (8-10) or "extreme" (11+)

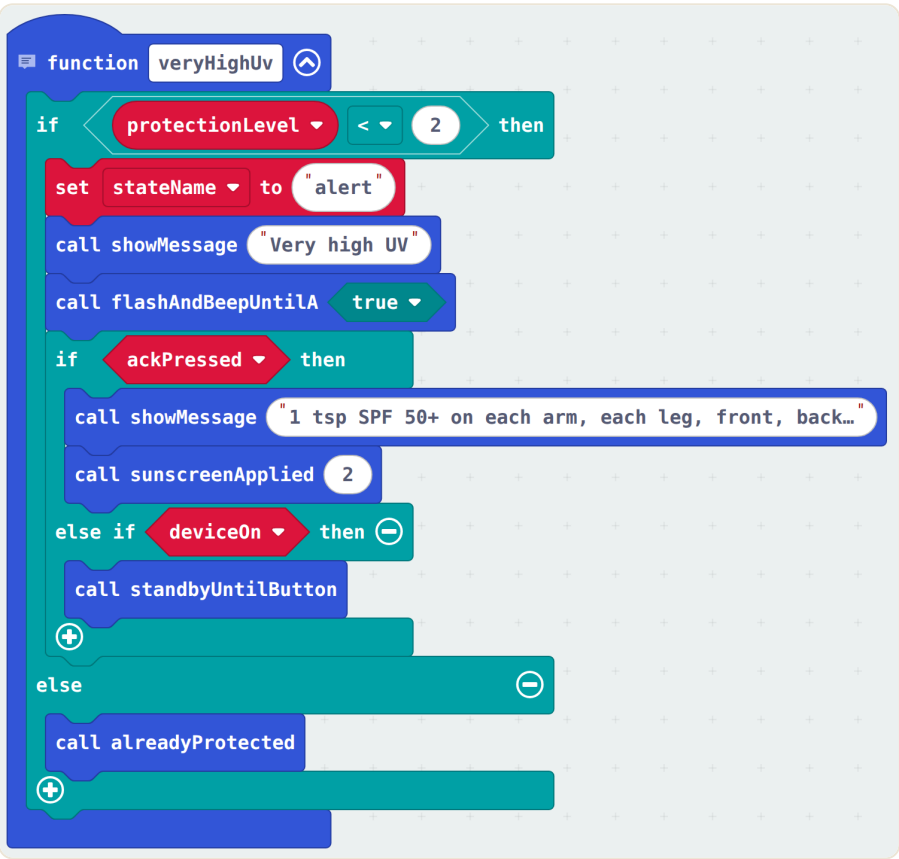
Used by: runFlowchart, sendStatus



moderateOrHighUv Function

Branch 3-7 moderate / high: Flash and slow beep until A is pressed, show how much sunscreen to put on, then start the 2 hour reapply timer. The flowchart comes back here every 5 minutes, so it only alerts while no sunscreen is on (no timer running). Otherwise it just shows the time left. If nobody presses A within alertGiveUpMinutes the device is probably not being worn, so it goes to standby until a button is pressed.

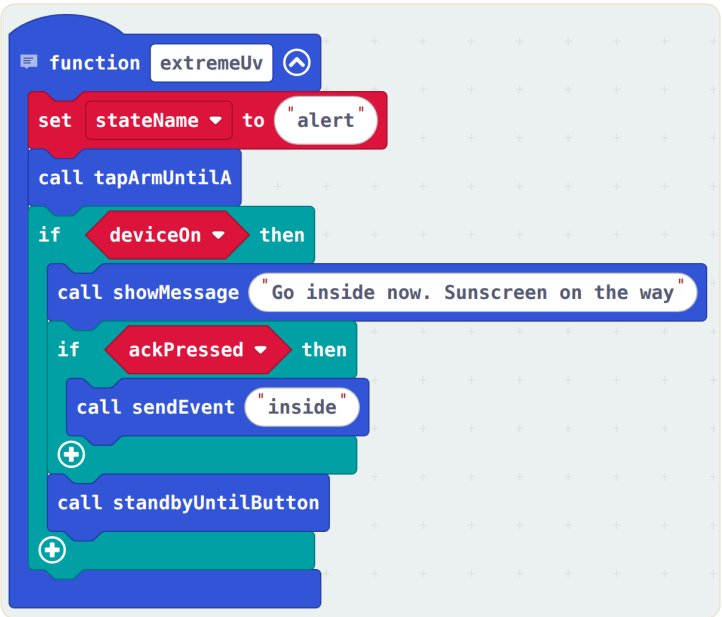
Used by: runFlowchart · **Uses:** sunscreenApplied, alreadyProtected, flashAndBeepUntilA, standbyUntilButton, showMessage



veryHighUv Function

Branch 8-10 very high: Flash and fast beep until A is pressed, sunscreen as above plus hat and shade, then start the 2 hour reapply timer. Alerts again if the wearer only had the moderate / high advice so far. No answer within alertGiveUpMinutes means standby, as above.

Used by: runFlowchart · **Uses:** sunscreenApplied, alreadyProtected, flashAndBeepUntilA, standbyUntilButton, showMessage



extremeUv Function

Branch 11+ extreme: Tap the arm until A is pressed (max 2 min, then beep instead), show go inside now, then standby until any button is pressed. The app only hears ev=inside when somebody actually pressed A.

Used by: runFlowchart · **Uses:** tapArmUntilA, standbyUntilButton, showMessage, sendEvent



reapplyTimerExpired Function that answers with a yes/no

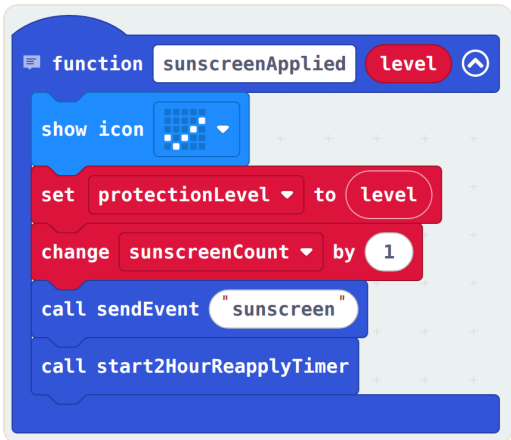
Decision: Reapply timer expired?

Used by: runFlowchart, wait5Minutes

2 The flowchart (continued)

20 blocks

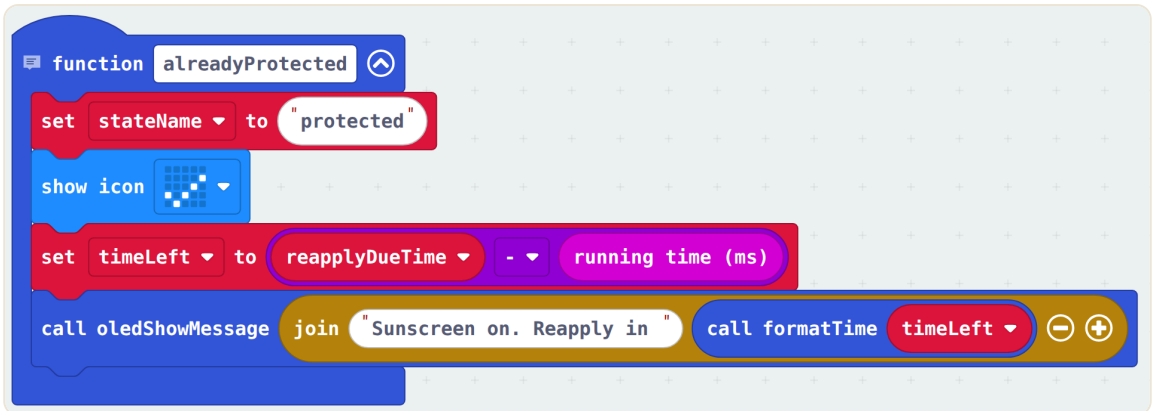
forever runs runFlowchart again and again. runFlowchart is the whole flowchart from top to bottom: read the UV sensor, sensor in range?, phone sending online UV?, UV index?, then one of the four branches, wait 5 minutes, reapply timer expired?. Every box and decision is a function with the same name, in the same order as the flowchart.



sunScreenApplied Function, inputs: level (number)

A was pressed after a sunscreen alert: remember the advice level, count it, tell the app, and start the timer

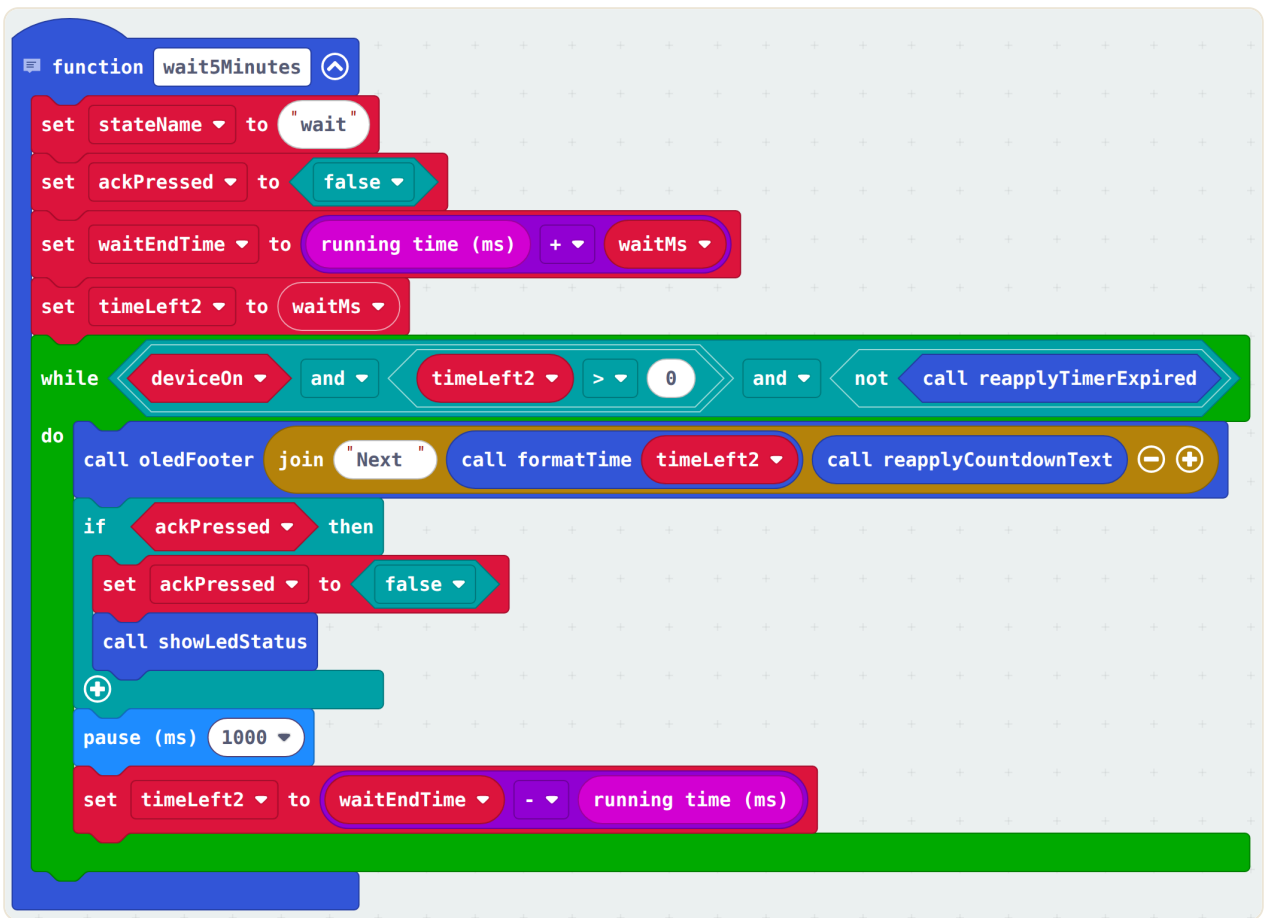
Used by: moderateOrHighUv, veryHighUv · **Uses:** start2HourReapplyTimer, sendEvent



alreadyProtected Function

Sunscreen is already on: no alert, just a tick and the time until reapply

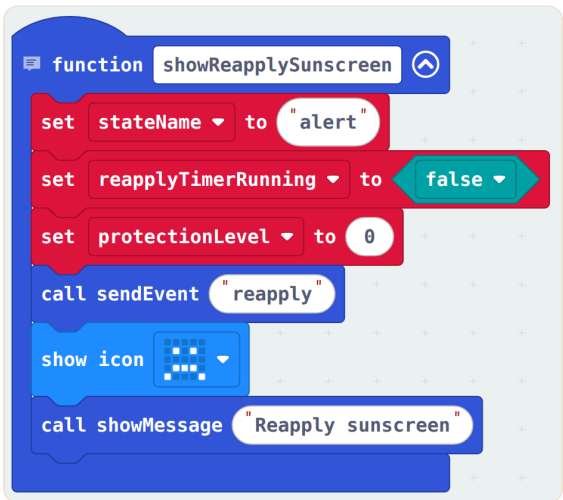
Used by: moderateOrHighUv, veryHighUv · **Uses:** oledShowMessage, formatTime



wait5Minutes Function

Box: Wait 5 minutes. Ends early if the device is turned off or the reapply timer runs out. Pressing A while waiting shows the UV index and the countdown.

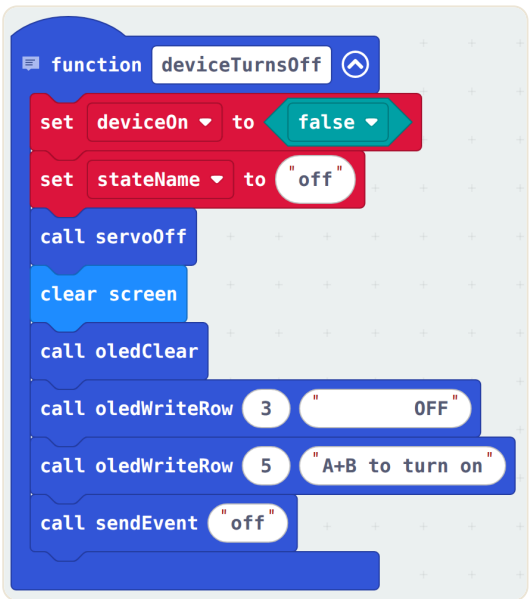
Used by: runFlowchart · **Uses:** reapplyTimerExpired, showLedStatus, oledFooter, formatTime, reapplyCountdownText



showReapplySunscreen Function

Box: Show reapply sunscreen. Clears the timer, so the next trip round the flowchart alerts again (if UV is still 3 or more) and starts a fresh 2 hours.

Used by: runFlowchart · **Uses:** showMessage, sendEvent



deviceTurnsOff Function

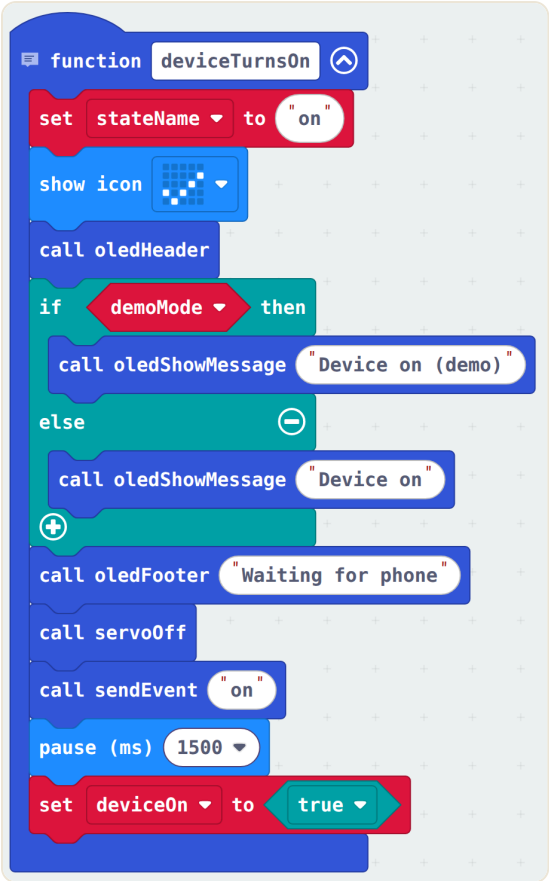
Terminal: Device turns off. Everything stops. Bluetooth stays up so the app can still turn it on again.

Used by: on button A+B pressed, handleCommand · **Uses:** servoOff, sendEvent, oledClear, oledWriteRow

2 The flowchart (continued)

20 blocks

forever runs runFlowchart again and again. runFlowchart is the whole flowchart from top to bottom: read the UV sensor, sensor in range?, phone sending online UV?, UV index?, then one of the four branches, wait 5 minutes, reapply timer expired?. Every box and decision is a function with the same name, in the same order as the flowchart.

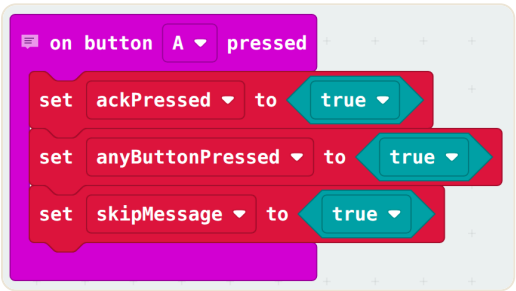


deviceTurnsOn Function
Terminal: Device turns on. Runs at start-up and when A+B turns it back on.
Used by: on button A+B pressed, handleCommand, on start · **Uses:** oledHeader, oledShowMessage, oledFooter, servoOff, sendEvent

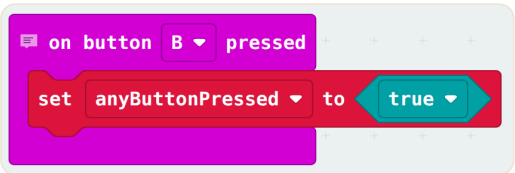
3 Buttons

3 blocks

A = done (sunscreen is on / going inside); A also skips a message scrolling on the LEDs, and while waiting it shows the UV index and the countdown. B = wake from standby. A+B = device turns off, or on again.



on button A pressed Event: runs each time A is pressed
A = "done": sunscreen is on / I am going inside. Also wakes from standby, skips a message scrolling on the LEDs, and while the device is waiting it shows the UV index and the countdown.



on button B pressed Event: runs each time B is pressed
B = wake from standby. (Hold B while switching on for demo timings.)



on button A+B pressed
Event: runs when A and B are pressed together
A + B = device turns off, or back on
Uses: deviceTurnsOn, deviceTurnsOff

Flash and beep, tap the arm with the servo, and standby. Each one keeps going until A is pressed (or the app sends "ack"), the device is turned off, or its time limit runs out. No answer within the time limit means the device is probably not being worn, so it goes to standby until a button is pressed.

```
function flashAndBeepUntilA fast
  set ackPressed to false
  set beatMs to 500
  set tone to 523
  if fast then
    set beatMs to 150
    set tone to 659
  +
  set giveUpTime to running time (ms) + alertGiveUpMs
  while call stillWaitingForA giveUpTime
  do
    call ledsAllOn
    call beep tone beatMs
    clear screen
    pause (ms) beatMs
```

flashAndBeepUntilA Function, inputs: fast (boolean)
Flash all the LEDs and beep until A is pressed (or the app sends "ack"). fast = true is the quicker, higher beep for very high UV. Gives up after alertGiveUpMinutes so a device left on a table does not beep all day; the caller then goes to standby.
Used by: moderateOrHighUv, veryHighUv, tapArmUntilA · **Uses:** stillWaitingForA, ledsAllOn, beep

```
function stillWaitingForA untilTime
  return not ackPressed and deviceOn and running time (ms) < untilTime
```

stillWaitingForA Function that answers with a yes/no, inputs: untilTime (number)
True while an alert should carry on: no A yet, device still on, time limit not reached
Used by: flashAndBeepUntilA, tapArmUntilA

```
function tapArmUntilA
  set ackPressed to false
  show icon
  set stopTappingTime to running time (ms) + tapMaxMs
  while call stillWaitingForA stopTappingTime
  do
    call servoTapOnce
  call servoOff
  if not ackPressed and deviceOn then
    call flashAndBeepUntilA true
  +
```

tapArmUntilA Function
The servo taps the arm until A is pressed. After tapMaxMinutes it beeps instead, so a device that is not being worn does not run the motor flat.
Used by: extremeUv · **Uses:** flashAndBeepUntilA, stillWaitingForA, servoOff, servoTapOnce

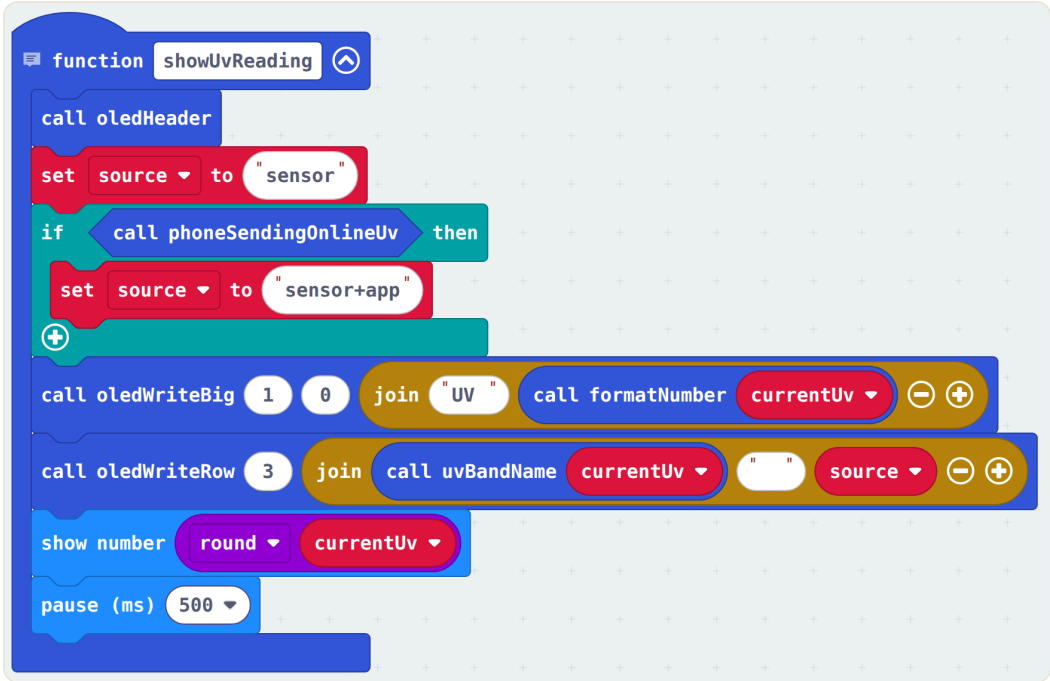
```
function standbyUntilButton
  set wentToStandby to true
  set stateName to "standby"
  call sendEvent "standby"
  set anyButtonPressed to false
  show icon
  call oledShowMessage "Standby. Press a button to check again"
  call oledFooter ""
  while deviceOn and not anyButtonPressed
  do
    pause (ms) 200
```

standbyUntilButton Function
Box: Standby until any button is pressed (A, B, or "ack" from the app). A+B still turns the device off. Afterwards the flowchart goes straight back to "Read UV sensor" (no 5 minute wait).
Used by: moderateOrHighUv, veryHighUv, extremeUv · **Uses:** oledShowMessage, oledFooter, sendEvent

5 Screen and LEDs

8 blocks

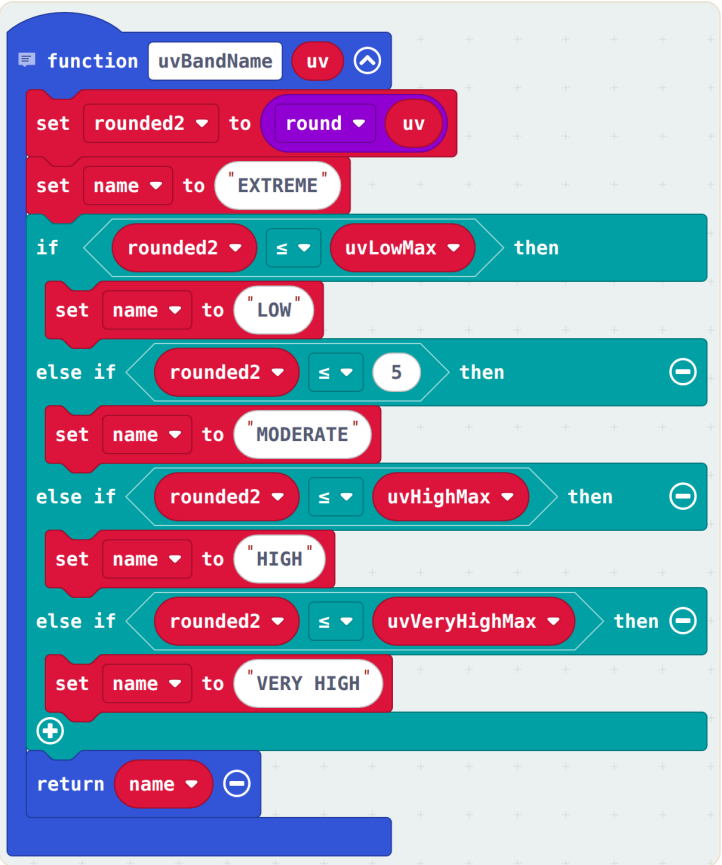
What the 5x5 LEDs show, and what the optional OLED screen shows: the UV number, the band, messages and the countdowns. Messages scroll across the LEDs when there is no OLED.



showUvReading Function

The rounded UV index on the LEDs; the exact number, band and source on the OLED

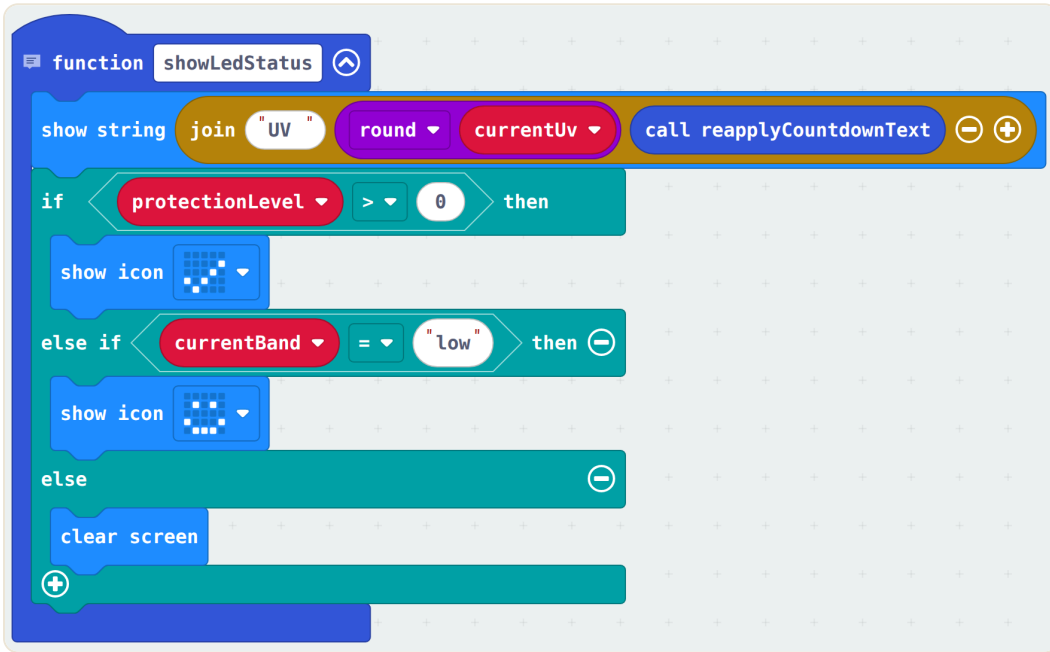
Used by: runFlowchart · **Uses:** phoneSendingOnlineUv, uvBandName, oledHeader, formatNumber, oledWriteRow, oledWriteBig



uvBandName Function that answers with a string, inputs: uv (number)

The words for the OLED: LOW, MODERATE, HIGH, VERY HIGH or EXTREME

Used by: showUvReading



showLedStatus Function

Pressing A while waiting: the UV index and the time until reapply scroll across the LEDs, then the icon comes back

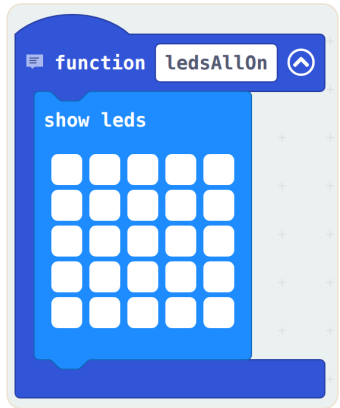
Used by: wait5Minutes · **Uses:** reapplyCountdownText



oledFooter Function, inputs: text (string)

OLED row 7: the countdowns

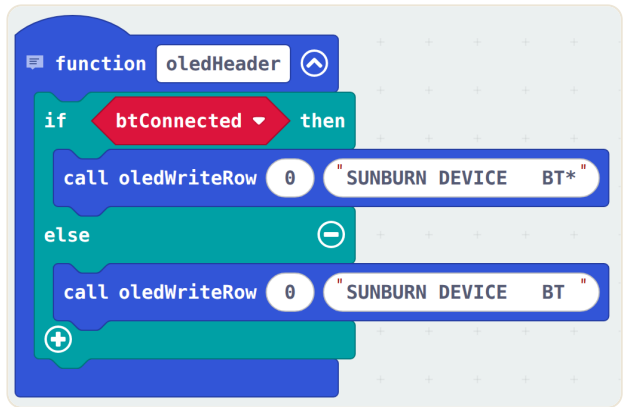
Used by: showCheckSensor, wait5Minutes, deviceTurnsOn, standbyUntilButton · **Uses:** oledWriteRow



ledsAllOn Function

All 25 LEDs on (the "flash")

Used by: flashAndBeepUntilA



oledHeader Function

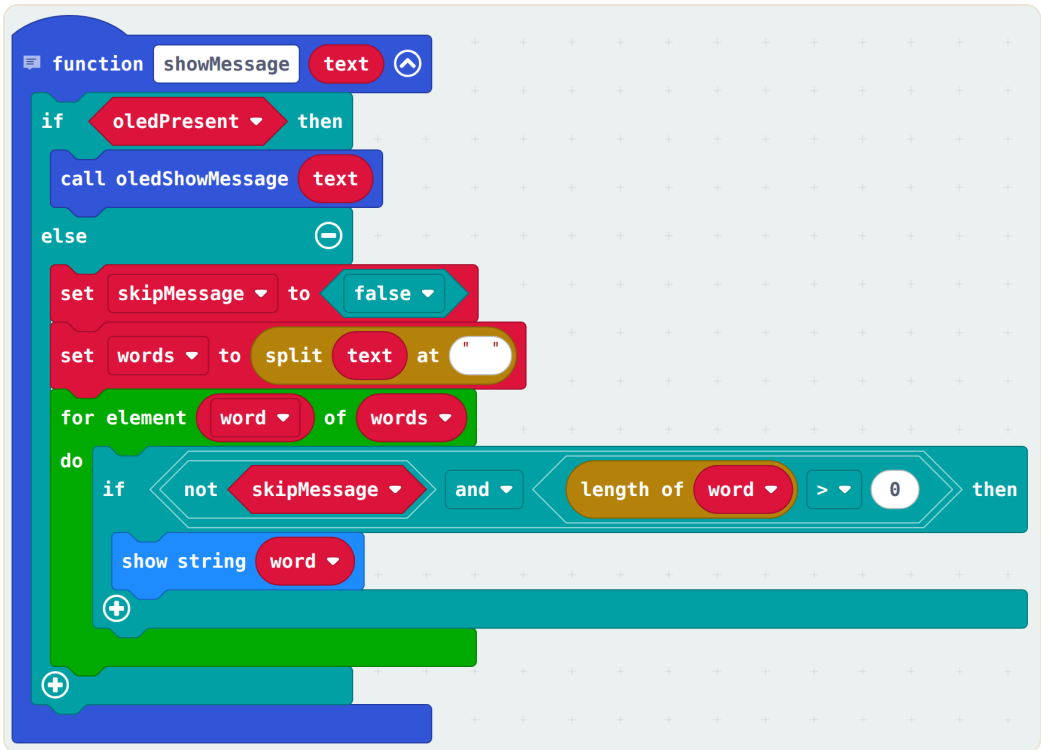
OLED row 0: the title and a Bluetooth star while a phone is connected

Used by: deviceTurnsOn, showUvReading, on bluetooth connected, on bluetooth disconnected · **Uses:** oledWriteRow

5 Screen and LEDs (continued)

8 blocks

What the 5x5 LEDs show, and what the optional OLED screen shows: the UV number, the band, messages and the countdowns. Messages scroll across the LEDs when there is no OLED.



showMessage Function, inputs: text (string)

A message: on the OLED if there is one, otherwise scrolled across the LEDs one word at a time. Pressing A skips the rest of a long message.

Used by: showCheckSensor, moderateOrHighUv, veryHighUv, extremeUv, showReapplySunscreen · **Uses:** oledShowMessage



oledShowMessage Function, inputs: text (string)

OLED rows 4-6: a message, word-wrapped over three lines of 21 characters

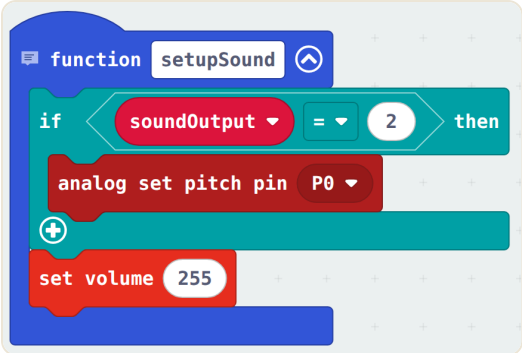
Used by: showHappyFace, alreadyProtected, deviceTurnsOn, standbyUntilButton, showMessage · **Uses:** oledWriteRow

6

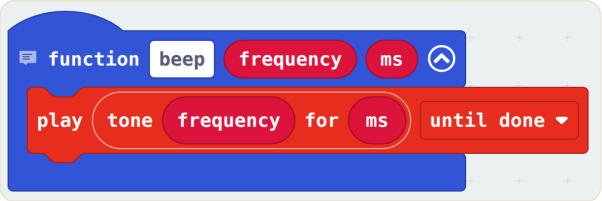
Sound and servo

5 blocks

Beeps on the V2 speaker (or a buzzer on P0), and the servo on P2 whose arm taps the wrist.



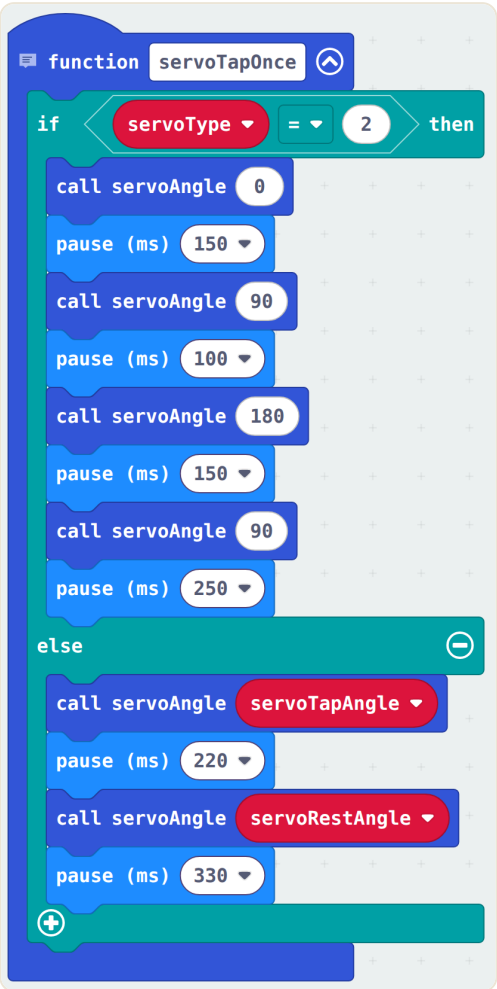
setupSound Function
V2 speaker, or a buzzer on P0
Used by: on start



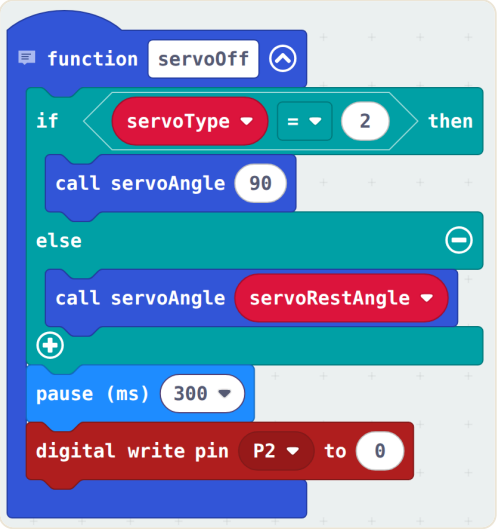
beep Function, inputs: frequency (number), ms (number)
One beep: frequency in Hz (523 = C5, 659 = E5, 440 = A4), length in ms
Used by: showCheckSensor, flashAndBeepUntilA



servoAngle Function, inputs: angle (number)
The servo signal wire is on P2
Used by: servoOff, servoTapOnce



servoTapOnce Function
One tap of the arm (about half a second)
Used by: tapArmUntilA · **Uses:** servoAngle



servoOff Function
Rest position, then stop the pulses so the servo is silent and uses no power
Used by: deviceTurnsOn, deviceTurnsOff, tapArmUntilA · **Uses:** servoAngle

7

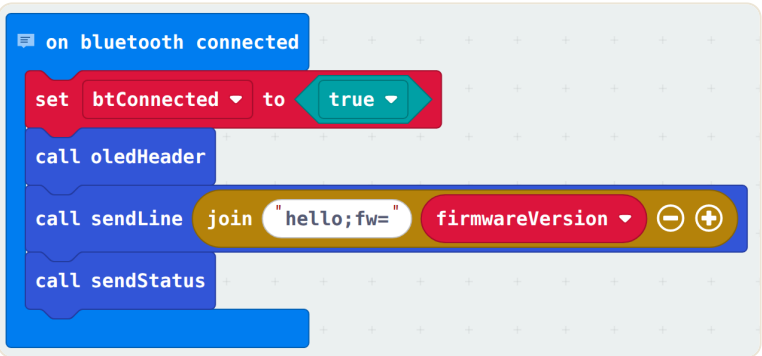
Bluetooth and serial

11 blocks

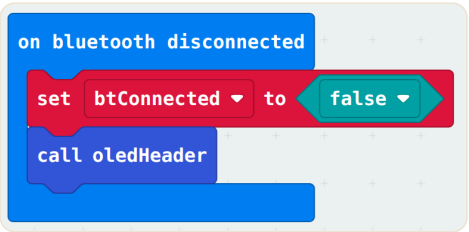
Talks to the phone app (Bluetooth UART) and to the USB serial console with the same text lines. Out every 2 s: st=wait;uv=7.3;sen=7.1;onl=6.5;band=vhigh;spf=5400;spfn=3;fw=3.1;demo=0. Out on events: ev=on, off, sunscreen, reapply, inside, standby, error. In, one per line: uv=6.5, ack, zero, cal=7, demo=1, power=0, debug=1, ping, read.



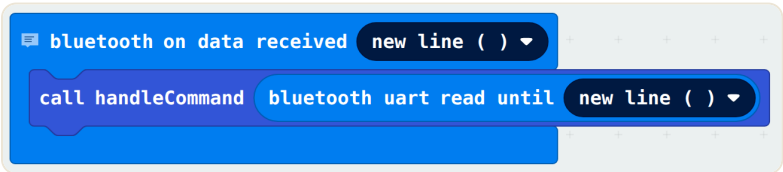
advertiseOverBluetooth Function
Box: Advertise over Bluetooth so a phone can connect (UART service). The same lines also go over USB serial, so a laptop can test everything without a phone.
Used by: on start



on bluetooth connected Event: a phone or computer connected
A phone connected: say hello and send the current status straight away
Uses: oledHeader, sendLine, sendStatus



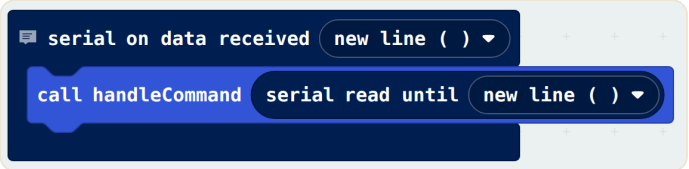
on bluetooth disconnected
Event: the phone went away
Uses: oledHeader



on bluetooth data received Event: a line of text arrived over Bluetooth
A command line arrived over Bluetooth
Uses: handleCommand



every 2000 ms Runs every 2 seconds in the background
Status line for the app every 2 seconds
Uses: sendStatus

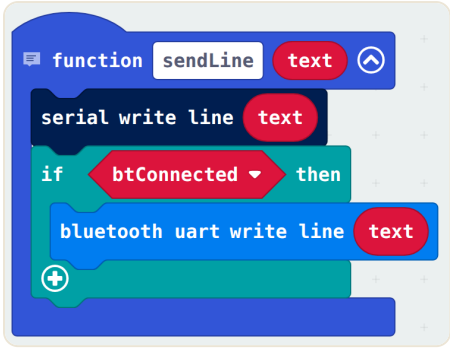


on serial data received Event: a line of text arrived over USB
A command line arrived over USB serial
Uses: handleCommand

7 Bluetooth and serial (continued)

11 blocks

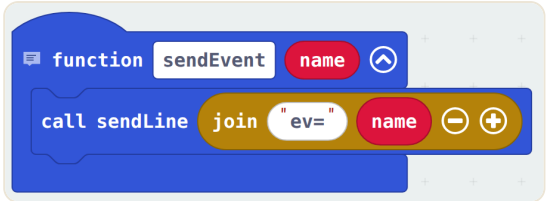
Talks to the phone app (Bluetooth UART) and to the USB serial console with the same text lines. Out every 2 s: st=wait;uv=7.3;sen=7.1;onl=6.5;band=vhigh;spf=5400;spfn=3;fw=3.1;demo=0. Out on events: ev=on, off, sunscreen, reapply, inside, standby, error. In, one per line: uv=6.5, ack, zero, cal=7, demo=1, power=0, debug=1, ping, read.



sendLine Function, inputs: text (string)

One line of text to USB serial, and to the phone when one is connected

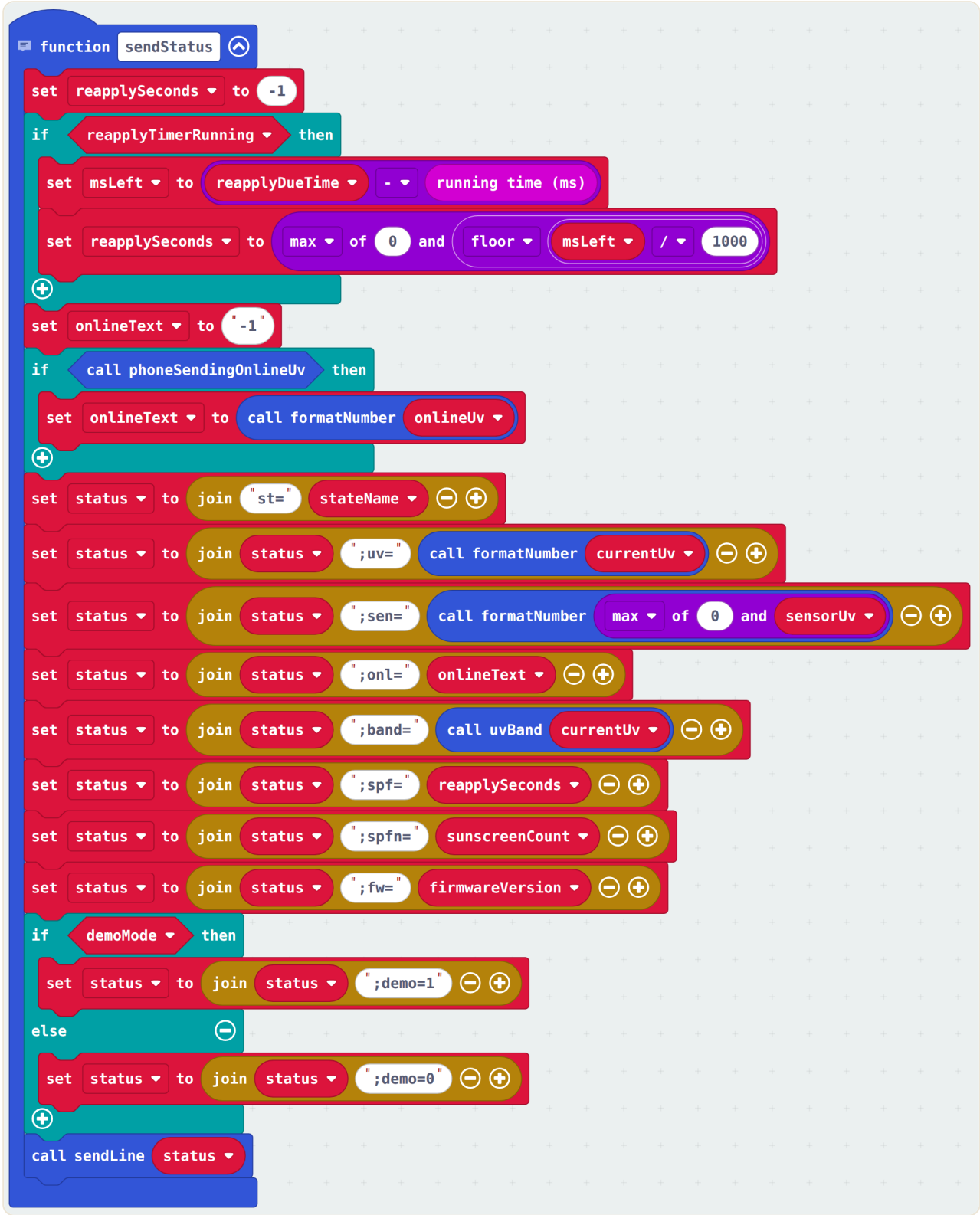
Used by: showCheckSensor, on bluetooth connected, sendEvent, sendStatus, handleCommand, calibrateToKnownUv



sendEvent Function, inputs: name (string)

Tells the app something happened, e.g. ev=sunscreen

Used by: extremeUv, sunscreenApplied, showReapplySunscreen, deviceTurnsOn, deviceTurnsOff, standbyUntilButton · **Uses:** sendLine



sendStatus Function

The status line: state, UV values, band, seconds until reapply, sunscreen count, firmware version, and whether demo timings are on

Used by: on bluetooth connected, every 2000 ms, handleCommand · **Uses:** phoneSendingOnlineUv, uvBand, sendLine, formatNumber

7 Bluetooth and serial (continued)

11 blocks

Talks to the phone app (Bluetooth UART) and to the USB serial console with the same text lines. Out every 2 s: st=wait;uv=7.3;sen=7.1;onl=6.5;band=vhigh;spf=5400;spfn=3;fw=3.1;demo=0. Out on events: ev=on, off, sunscreen, reapply, inside, standby, error. In, one per line: uv=6.5, ack, zero, cal=7, demo=1, power=0, debug=1, ping, read.

```
function handleCommand command
  set parts to split call cleanText command at "="
  set key to parts get value at 0
  if length of array parts > 1 then
    set value to parts get value at 1
    call debugLog join "cmd" key " " value
    if key = "uv" then
      set newUv to parse to number value
      if newUv ≥ 0 and newUv ≤ 20 then
        set onlineUv to newUv
        set onlineUvTime to running time (ms)
        call sendLine "ok=uv"
      else
        call sendLine "err=uv"
    else if key = "ack" then
      set ackPressed to true
      set anyButtonPressed to true
      call sendLine "ok=ack"
    else if key = "zero" then
      call readUvSensor
      set uvZeroVolts to sensorVolts
      call sendLine join "ok=zero;v=" call formatNumber uvZeroVolts
    else if key = "cal" then
      call calibrateToKnownUv parse to number value
    else if key = "demo" then
      set demoMode to value = 1
      call applyTimings
      call sendLine "ok=demo"
    else if key = "power" then
      if value = 0 and deviceOn then
        call deviceTurnsOff
      else if value = 1 and not deviceOn then
        call deviceTurnsOn
    else if key = "debug" then
      set debugMode to value = 1
      call sendLine "ok=debug"
    else if key = "ping" then
      set oledText to 0
      if oledPresent then
        set oledText to 1
      call sendLine join "pong;fw=" firmwareVersion ";oled=" oledText
    else if key = "read" then
      call sendStatus
    else if length of key > 0 then
      call sendLine "err=unknown"
```

handleCommand Function, inputs: command (string)

One command per line from the app or the serial console: "key=value" or just "key"

Used by: on bluetooth data received, on serial data received · **Uses:** readUvSensor, deviceTurnsOn, deviceTurnsOff, sendLine, sendStatus, calibrateToKnownUv, applyTimings, formatNumber, cleanText, debugLog

```
function calibrateToKnownUv knownUv
  call readUvSensor
  if knownUv > 0 and sensorVolts > uvZeroVolts then
    set uvVoltsPerIndex to sensorVolts - uvZeroVolts / knownUv
    call sendLine "ok=cal"
  else
    call sendLine "err=cal"
```

calibrateToKnownUv Function, inputs: knownUv (number)

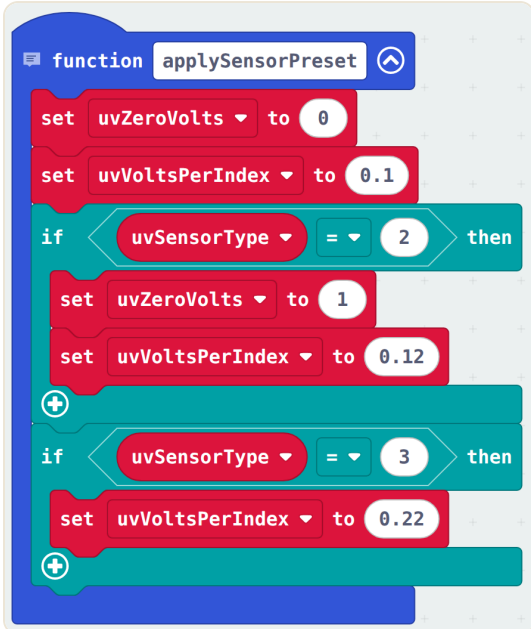
Live calibration: send cal=7 while a weather site says the UV index is 7 and the sensor is in the sun. Send "zero" in the dark first if needed.

Used by: handleCommand · **Uses:** readUvSensor, sendLine

8 Helpers

10 blocks

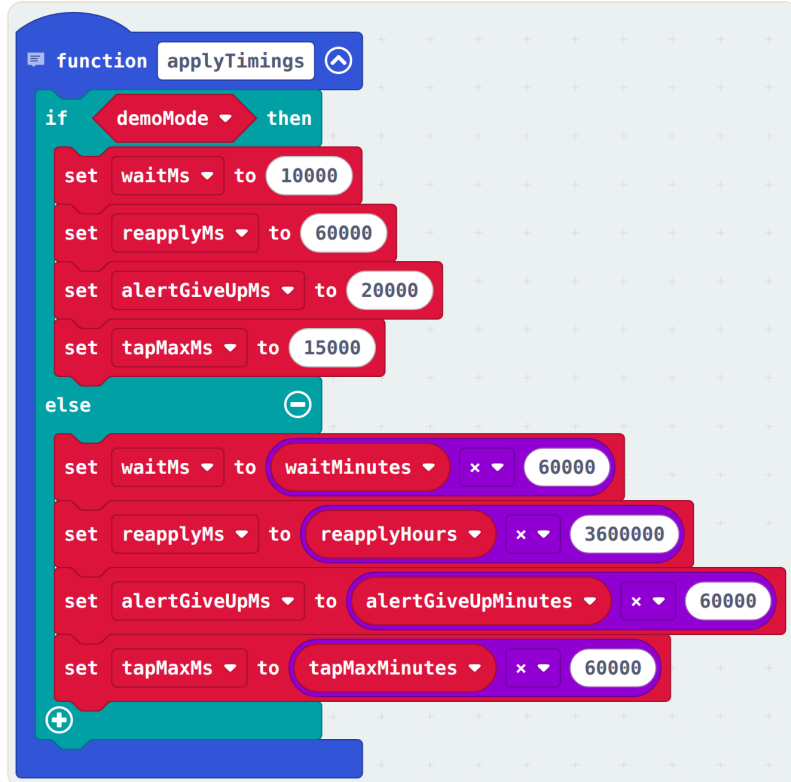
Small tools used by the rest: sensor presets, the timings, sorting, number and time formatting, tidying up commands, and the debug lines (send debug=1 to see them).



applySensorPreset Function

The maths for each sensor type: UV index = (volts - uvZeroVolts) / uvVoltsPerIndex

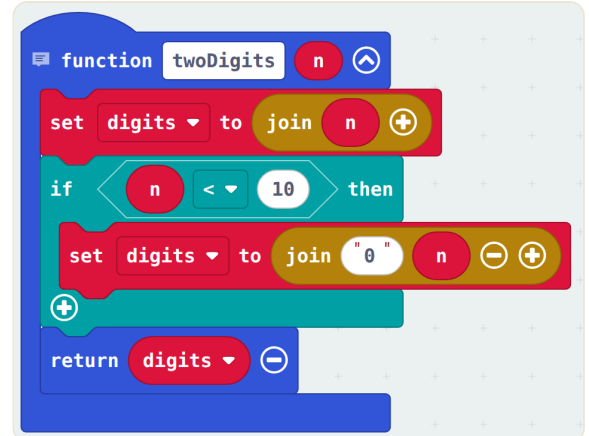
Used by: on start



applyTimings Function

Turns the minutes and hours in the settings into milliseconds. Demo mode uses fast timings so the whole flowchart can be shown in a few minutes.

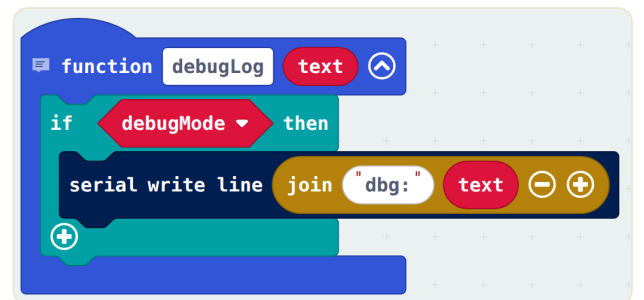
Used by: handleCommand, on start



twoDigits Function that answers with a string, inputs: n (number)

7 -> "07", 42 -> "42"

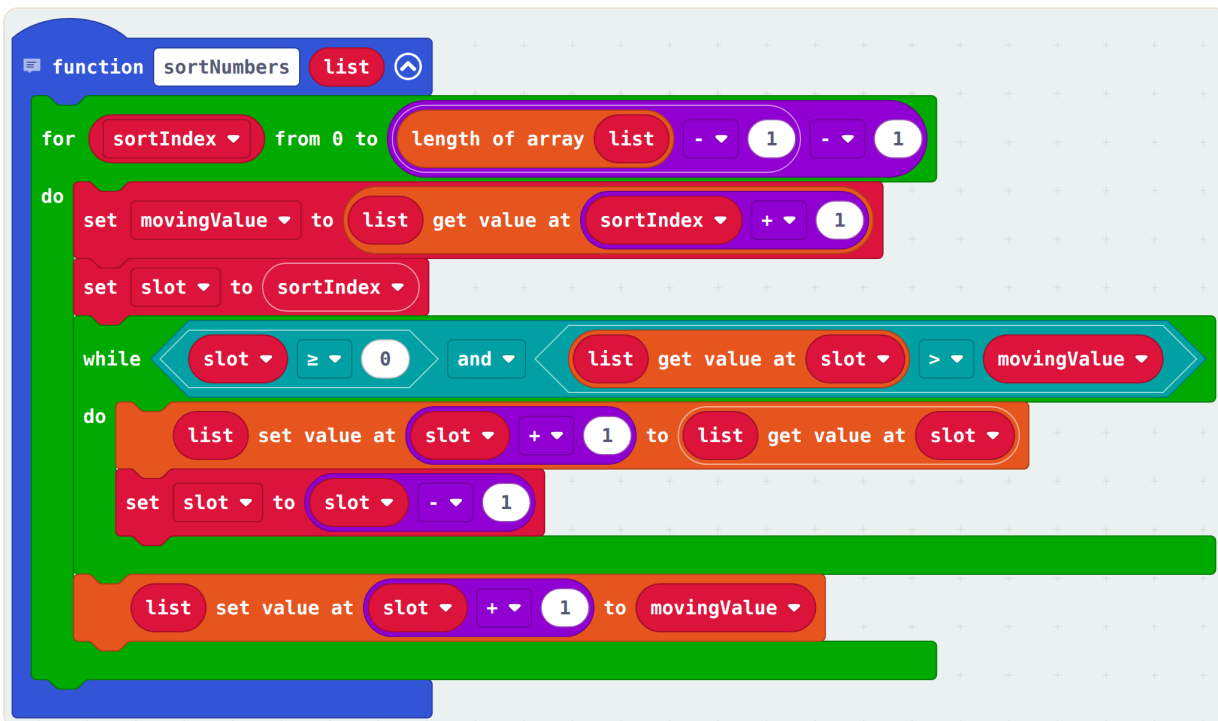
Used by: formatTime



debugLog Function, inputs: text (string)

Extra "dbg:" lines on the serial console after the command debug=1

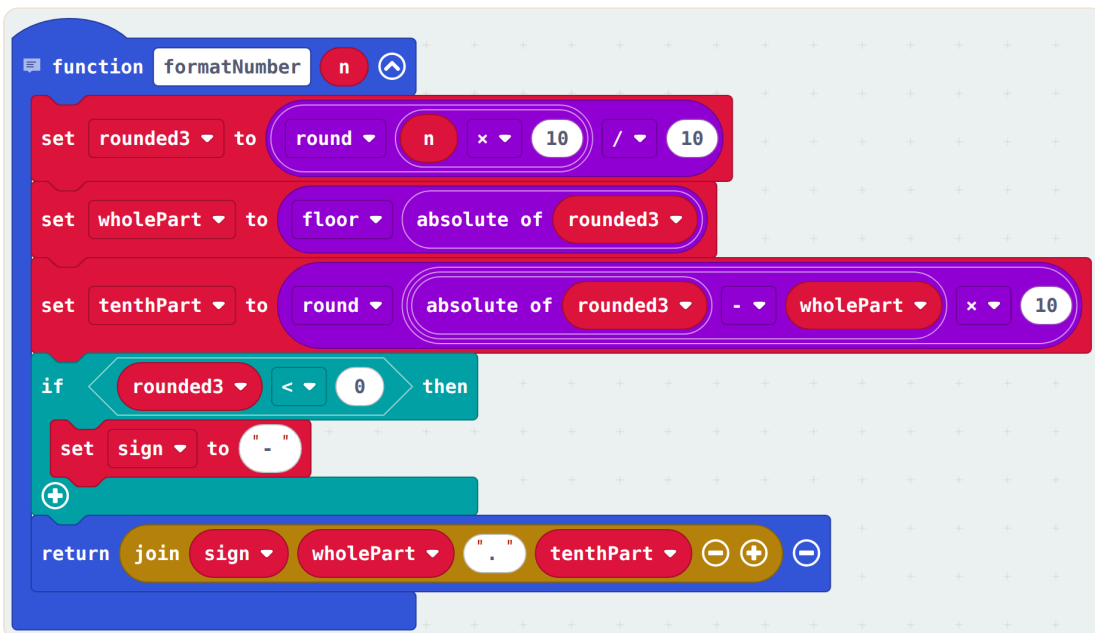
Used by: readUvSensor, handleCommand



sortNumbers Function, inputs: list (number[])

Sorts a list of numbers from smallest to largest (insertion sort)

Used by: readUvSensor



formatNumber Function that answers with a string, inputs: n (number)

A number with one decimal place: 7.25 -> "7.3"

Used by: showCheckSensor, showUvReading, sendStatus, handleCommand

8 Helpers (continued)

10 blocks

Small tools used by the rest: sensor presets, the timings, sorting, number and time formatting, tidying up commands, and the debug lines (send debug=1 to see them).

```
function formatTime ms
  set totalSeconds to floor max of 0 and ms / 1000
  set hours to floor totalSeconds / 3600
  set minutes to remainder of floor totalSeconds / 60 / 60
  set seconds to remainder of totalSeconds / 60
  if hours = 0 then
    set timeText to join minutes ":" call twoDigits seconds
  else
    set timeText to join hours "h" call twoDigits minutes
  return timeText
```

formatTime Function that answers with a string, inputs: ms (number)
Milliseconds as a clock: 90000 -> "1:30", 7200000 -> "2h00"
Used by: alreadyProtected, wait5Minutes, reapplyCountdownText · **Uses:** twoDigits

```
function reapplyCountdownText
  if reapplyTimerRunning then
    set timeLeft3 to reapplyDueTime - running time (ms)
    set countdown to join " SPF " call formatTime timeLeft3
  return countdown
```

reapplyCountdownText Function that answers with a string
" SPF 1h23" for the OLED footer while the reapply timer is running
Used by: wait5Minutes, showLedStatus · **Uses:** formatTime

```
function cleanText text
  set cleaned to text
  set lastCode to char code from cleaned at length of cleaned - 1
  while length of cleaned > 0 and call isBlank lastCode
    do
      set cleaned to substring of cleaned from 0 of length length of cleaned - 1
      set lastCode to char code from cleaned at length of cleaned - 1
  while length of cleaned > 0 and call isBlank char code from cleaned at 0
    do
      set cleaned to substring of cleaned from 1 of length length of cleaned - 1
  return cleaned
```

cleanText Function that answers with a string, inputs: text (string)
Removes spaces and line endings from both ends of a command
Used by: handleCommand · **Uses:** isBlank

```
function isBlank code
  return code = 32 or code = 13 or code = 10
```

isBlank Function that answers with a yes/no, inputs: code (number)
Character codes 32 = space, 13 = return, 10 = new line
Used by: cleanText

9

OLED screen driver (optional)

12 blocks

Only used if you add a 128x64 I2C screen (SSD1306 or SH1106) on P19/P20. Draws text on it without needing an extension. The program detects the screen at start-up and works exactly the same without one.

```
function setupOled
  set oledPresent to false
  if oledMode > 0 then
    call loadFont
    call buildNibbleTable
    call oledCommand 174
    set screenStatus to i2c read number at address oledAddress of format UInt8LE repeated false
    if oledMode = 2 or screenStatus = 64 then
      call oledInitCommands
      set oledPresent to true
      call oledClear
```

setupOled Function
Looks for a screen and sets it up. The program works exactly the same without one.
Used by: on start · **Uses:** oledInitCommands, loadFont, buildNibbleTable, oledCommand, oledClear

```
function loadFont
  set hex to join hex "000000000000005f00000007000700147f147f14242a7f2a..."
  set hex to join hex "21c0014083e081408083e08080050300000080808080006..."
  set hex to join hex "2141454b311814127f1027454545393c4a49493001710905..."
  set hex to join hex "24100141414141400412214080201510906324979413e7e1..."
  set hex to join hex "7f090909013e4149497a7f0808087f00417f41002040413f..."
  set hex to join hex "1413e7f090909063e4151215e7f0919294646494931010..."
  set hex to join hex "07087008076151494543007f41410002040810200041417f..."
  set hex to join hex "444383844444420384444487f3854545418087e0901020c5..."
  set hex to join hex "00417f40007c041804787c0804047838444444387c141414..."
  set hex to join hex "0207c1c2040201c3c4030403c44281028440c5050503c446..."
  set hexDigits to "0123456789abcdef"
  set fontBytes to empty array
  for fontIndex from 0 to 474
    do
      set highDigit to hexDigits find index of char from hex at fontIndex x 2
      set lowDigit to hexDigits find index of char from hex at fontIndex x 2 + 1
      fontBytes add value highDigit x 16 + lowDigit to end
```

loadFont Function
A 5x7 font for characters 32-126, stored as hex text and unpacked into fontBytes
Used by: setupOled

```
function oledInitCommands
  call oledCommand 213
  call oledCommand 128
  call oledCommand 168
  call oledCommand 63
  call oledCommand 211
  call oledCommand 0
  call oledCommand 64
  if oledType = 2 then
    call oledCommand 173
    call oledCommand 139
  else
    call oledCommand 141
    call oledCommand 20
    call oledCommand 32
    call oledCommand 2
  call oledCommand 161
  call oledCommand 200
  call oledCommand 218
  call oledCommand 18
  call oledCommand 129
  call oledCommand 207
  call oledCommand 217
  call oledCommand 241
  call oledCommand 219
  call oledCommand 64
  call oledCommand 164
  call oledCommand 166
  call oledCommand 175
```

oledInitCommands Function
The standard start-up sequence for SSD1306 and SH1106 screens
Used by: setupOled · **Uses:** oledCommand

```
function oledCommand command
  i2c write number
  at address oledAddress
  with value command
  of format UInt16BE
  repeated false
```

oledCommand Function, inputs:
command (number)
Sends one command byte (control byte 0, then the command)
Used by: setupOled, oledInitCommands, oledSetPosition

9

OLED screen driver (optional) (continued)

12 blocks

Only used if you add a 128x64 I2C screen (SSD1306 or SH1106) on P19/P20. Draws text on it without needing an extension. The program detects the screen at start-up and works exactly the same without one.

```
function buildNibbleTable
  set nibbleStretch to empty array
  for nibble from 0 to 15
    do
      set remaining to nibble
      set weight to 3
      repeat 4 times
        do
          if remainder of remaining / 2 = 1 then
            change stretched by weight
          +
          set remaining to floor remaining / 2
          set weight to weight x 4
        do
          nibbleStretch add value stretched to end
```

buildNibbleTable Function
Table for double-size text: each of the 4 bits in a nibble is doubled (0101 -> 00110011)
Used by: setupOled

```
function oledSetPosition page col
  set pixelColumn to col
  if oledType = 2 then
    change pixelColumn by 2
  +
  call oledCommand 176 + page
  call oledCommand remainder of pixelColumn / 16
  call oledCommand 16 + floor pixelColumn / 16
```

oledSetPosition Function, inputs: page (number), col (number)
Moves the cursor: page = 8-pixel row (0-7), col = pixel column (0-127)
Used by: oledClear, oledWriteText, oledWriteBig · **Uses:** oledCommand

```
function oledWriteRow row text
  set rowText to text
  while length of rowText < 21
    do
      set rowText to join rowText " "
  +
  call oledWriteText row 0 rowText
```

oledWriteRow Function, inputs: row (number), text (string)
Writes a whole row, padding with spaces so old text is wiped
Used by: deviceTurnsOff, showUvReading, oledHeader, oledShowMessage, oledFooter · **Uses:** oledWriteText

```
function fontByte code column
  set safeCode to code
  if safeCode < 32 or safeCode > 126 then
    set safeCode to 63
  +
  return fontBytes get value at safeCode - 32 x 5 + column
```

fontByte Function that answers with a number, inputs: code (number), column (number)
One column (0-4) of one character
Used by: oledWriteText, oledWriteBig

```
function oledData byte1 byte2 byte3
  set packet to byte1 x 65536
  change packet by byte2 x 256
  change packet by byte3
  i2c write number
  at address oledAddress
  with value 1073741824 + packet
  of format UInt32BE
  repeated false
```

oledData Function, inputs: byte1 (number), byte2 (number), byte3 (number)
Sends three pixel-data bytes (control byte 64, then the data)
Used by: oledClear, oledWriteText, oledWriteBig

```
function oledClear
  if oledPresent then
    for clearPage from 0 to 7
      do
        call oledSetPosition clearPage 0
        repeat 43 times
          do
            call oledData 0 0 0
      +
    +
```

oledClear Function
Blanks the whole screen
Used by: deviceTurnsOff, setupOled · **Uses:** oledData, oledSetPosition

9

OLED screen driver (optional) (continued)

12 blocks

Only used if you add a 128x64 I2C screen (SSD1306 or SH1106) on P19/P20. Draws text on it without needing an extension. The program detects the screen at start-up and works exactly the same without one.

```
function oledWriteText row col text
  if oledPresent then
    set smallText to text
    if length of smallText > 21 - col then
      set smallText to substring of smallText from 0 of length 21 - col
    call oledSetPosition row col x 6
    for charIndex from 0 to length of smallText - 1
      do
        set charCode to char code from smallText at charIndex
        call oledData call fontByte charCode 0 call fontByte charCode 1 call fontByte charCode 2
        call oledData call fontByte charCode 3 call fontByte charCode 4 0
```

oledWriteText Function, inputs: row (number), col (number), text (string)

Small text: 21 characters per row, 8 rows

Used by: oledWriteRow · **Uses:** fontByte, oledData, oledSetPosition

9 OLED screen driver (optional) (continued)

12 blocks

Only used if you add a 128x64 I2C screen (SSD1306 or SH1106) on P19/P20. Draws text on it without needing an extension. The program detects the screen at start-up and works exactly the same without one.

```
function oledWriteBig row col text
  if oledPresent then
    set bigText to text
    if length of bigText > 10 then
      set bigText to substring of bigText from 0 of length 10
    for half from 0 to 1
      do
        call oledSetPosition row + half col * 6
        for bigCharIndex from 0 to length of bigText - 1
          do
            set bigCharCode to char code from bigText at bigCharIndex
            for columnIndex from 0 to 5
              do
                if columnIndex < 5 then
                  set glyphColumn to call fontByte bigCharCode columnIndex
                if half = 0 then
                  stretchedColumns add value nibbleStretch get value at remainder of glyphColumn / 16 to end
                else
                  stretchedColumns add value nibbleStretch get value at floor glyphColumn / 16 to end
            call oledData stretchedColumns get value at 0 stretchedColumns get value at 0 stretchedColumns get value at 1
            call oledData stretchedColumns get value at 1 stretchedColumns get value at 2 stretchedColumns get value at 2
            call oledData stretchedColumns get value at 3 stretchedColumns get value at 3 stretchedColumns get value at 4
            call oledData stretchedColumns get value at 4 stretchedColumns get value at 5 stretchedColumns get value at 5
```

oledWriteBig Function, inputs: row (number), col (number), text (string)

Double-size text: two rows tall, up to 10 characters

Used by: showUvReading · **Uses:** fontByte, oledData, oledSetPosition